A Corrected Metatheory for Mergeable Replicated Data Types

Anonymous Authors

Abstract

Mergeable replicated data types let users update independent branches and reconcile them with a three-way merge. Their correctness argument must explain the state of every stored version, including old versions that later become merge bases. We show that a published induction for this purpose is unsound: an event outside the version under consideration can remove an ordering constraint that the version still needs. We mechanize a counterexample and a corrected proof in Lean 4. The correction establishes a unique replay result for each stored event set, then relates that internal result to an independent single-user specification that checks which operations clients may create and what queries must return. We also verify local collection of asynchronously fetched version histories. A replica may discard the root and intermediate ancestor paths while preserving the reachability and merge-base answers that future execution needs. This collector composes with datatype-specific removal of metadata inside retained states. Together, the results give a machine-checked account of execution, client-visible correctness, and history reclamation, with an explicit boundary around the unverified runtime.

Contents

1 Problem, result, and evidence boundary **2**

2 Raw MRDT semantics **3**
2.1 The datatype signature . . . 3
2.2 Configurations and transitions . . . 3

3 Why the earlier induction fails **4**

4 Internal replay adequacy **6**
4.1 Canonical states . . . 6
4.2 The ternary Join obligation . . . 7
4.3 One reusable proof method for all-context Join . . . 8

5 Issuance, sequential meaning, and safety **10**

6 Canonical virtual merge bases **12**

7 Distributed commit-history garbage collection **13**
7.1 Minimal local state . . . 13
7.2 Collection certificate . . . 13
7.3 Direct asynchronous refinement . . . 14

8 Optional datatype-state garbage collection **15**

9 Machine-checked instances **16**

10 Related work **18**

11 Reproduction and residual boundary **18**

12 Conclusion **18**

1 Problem, result, and evidence boundary

An MRDT stores immutable versions in a Git-like directed acyclic graph (DAG) [1]. An update extends one replica head. A merge of branch states a and b reads the state l at their common ancestor and computes $\mathsf{merge}(l, a, b)$. The state at each registered version should match some single-user ordering of exactly that version's operations, and its queries should agree with the datatype's independent sequential specification.

Figure 1 is the running execution shape. The graph is not decoration: apply creates a one-parent version, merge creates a two-parent version, and a later merge may use an old version as its base. This is why the correctness invariant ranges over the store rather than only the current heads.

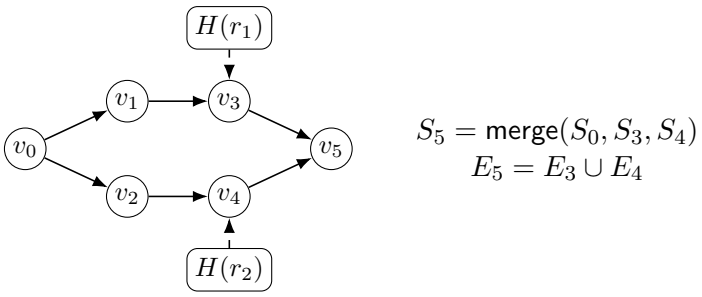


Figure 1: A version-DAG execution. Solid arrows point from parents to children. The merge at v_5 reads the registered GCA v_0 and both branch heads.

The development has three results. First, it repairs the replay induction and extends the proof to histories with several maximal common ancestors. Second, it connects those internal replay witnesses to legal histories and observations of an independent sequential datatype. Third, it proves that two forms of collection preserve the uncollected execution: one removes obsolete versions, and the other removes datatype metadata inside a retained version.

This boundary matters. A theorem about a semantic MRDT does not by itself verify an editor widget, a wire decoder, or crash recovery. The JavaScript runtime is tested against the model-facing fixtures, but it is not extracted from Lean. Its evidence manifest names the exact Lean package associated with each released datatype and labels representation experiments separately.

Evidence convention. We call a proposition accepted by Lean *machine-checked*, an executable runtime correspondence test *validated*, and a benchmark observation *measured*. The trusted model includes Lean's kernel, Mathlib definitions, and the paper-to-source transcription. It does not include the JavaScript runtime: that implementation is not extracted from Lean. The accompanying claim ledger records a source declaration for every load-bearing statement.

2 Raw MRDT semantics

2.1 The datatype signature

Definition 2.1 (MRDT signature)**.** An MRDT signature D contains a state type Σ, an initial state σ_0, application-operation type O, query type Q, result type R, and total functions

$$\mathsf{do} : \Sigma \to (\mathbb{N} \times \mathbb{N} \times O) \to \Sigma, \quad \mathsf{query} : \Sigma \to Q \to R, \quad \mathsf{merge} : \Sigma \to \Sigma \to \Sigma \to \Sigma.$$

An event $e = (t, r, o)$ contains a globally unique Lamport timestamp, replica identifier, and application operation. The execution semantics requires e_1 vis e_2 to imply $t(e_1) < t(e_2)$. In $\mathsf{merge}(l, a, b)$, l is the common-ancestor state and a, b are the branch states. This function handles every merge. `MRDTSig`

These functions define the datatype-level computation. The next section lifts them to executions over version DAGs.

2.2 Configurations and transitions

A configuration contains the semantically relevant state

$$C = \langle S, H, P, \mathsf{vis} \rangle, \qquad \mathit{Timestamp} = \mathit{Replica} = \mathit{Version} = \mathbb{N},$$
$$\mathit{Event} = \mathit{Timestamp} \times \mathit{Replica} \times O,$$
$$S : \mathit{Version} \rightharpoonup \big(\Sigma \times \mathcal{P}(\mathit{Event})\big), \qquad H : \mathit{Replica} \rightharpoonup \mathit{Version},$$
$$P : \mathit{Version} \to \mathsf{List}(\mathit{Version}), \qquad \mathsf{vis} \subseteq \mathit{Event} \times \mathit{Event},$$
$$\mathsf{state} : \mathsf{dom}(S) \to \Sigma, \qquad \mathcal{E} : \mathsf{dom}(S) \to \mathcal{P}(\mathit{Event}), \qquad S(v) = \big(\mathsf{state}(v), \mathcal{E}(v)\big) \quad (v \in \mathsf{dom}(S)).$$

where S is a partial version store, H maps replicas to heads, P maps a version to its parents, and vis is event visibility. Every finite execution allocates only finitely many entries of S. The functions state and $\mathcal{E}$ are its state and event-set projections on allocated versions. If $H(r) = v$, the state and event set at active replica r are $\mathsf{state}(v)$ and $\mathcal{E}(v)$.

A valid configuration also satisfies the following conditions:

1. every visibility endpoint occurs in an active event set, visibility is causally closed at each active replica, and same-replica events are visible in one direction;
2. distinct stored events have distinct timestamps, and e_1 vis e_2 implies $t(e_1) < t(e_2)$;
3. parent versions have smaller version numbers, version 0 stores $(\sigma_0, \varnothing)$, and every active head names a stored state and event set;
4. if v_T is a greatest common ancestor (GCA) of stored v_1, v_2, then

$$\mathcal{E}(v_T) = \mathcal{E}(v_1) \cap \mathcal{E}(v_2).$$

The last condition is part of this semantic model, rather than a property of an arbitrary Git DAG. The transition rules preserve it.

If $H(r) = v$, applying a fresh event $e = (t, r, o)$ extends visibility from every event at the issuer head to e:

$$\mathsf{vis}' = \mathsf{vis} \ \cup \big(\mathcal{E}(v) \times \{e\}\big).$$

Merge does not add an event and therefore leaves visibility unchanged.

Figure 2 gives the four transition forms. The notation $C[v \mapsto (s, E); r \mapsto v; P(v) \mapsto \pi]$ updates the version store, replica head, and parent list while preserving all other entries. The raw Apply

rule accepts any application operation with a fresh timestamp; Section 5 later restricts which operations a client may create. Write $\mathsf{freshTime}(C, t)$ when no event at an active replica or stored version has timestamp t. Checking the store matters because an old version need not remain an active head. Because the resulting configuration must also satisfy causal timestamp monotonicity, every event at the issuer head has timestamp less than t.

$$\textsc{Fork}\quad \frac{H(r_s) = v \quad H(r_d) = \bot \quad v' \notin \mathsf{dom}(S) \quad v < v'}{C \xrightarrow{\mathsf{fork}(r_d, r_s)} C[v' \mapsto S(v); r_d \mapsto v'; P(v') \mapsto [v]]}$$

$$\textsc{Query}\quad \frac{H(r) = v \quad S(v) = (s, E) \quad z = \mathsf{query}(s, q)}{C \xrightarrow{\mathsf{query}(r,q,z)} C}$$

$$\textsc{Apply}\quad \frac{H(r) = v \quad S(v) = (s, E) \quad \mathsf{freshTime}(C, t) \quad v' \notin \mathsf{dom}(S) \quad v < v' \quad e = (t, r, o)}{C \xrightarrow{\mathsf{apply}(t,r,o)} C[v' \mapsto (\mathsf{do}(s, e), E \cup \{e\}); r \mapsto v'; P(v') \mapsto [v]]}$$

$$\textsc{Merge}\quad \frac{H(r_1) = v_1 \quad H(r_2) = v_2 \quad \mathsf{GCA}_P(v_1, v_2) = v_T \quad S(v_i) = (s_i, E_i)\ (i \in \{1, 2, T\}) \quad v_m \notin \mathsf{dom}(S) \quad v_1, v_2 < v_m}{C \xrightarrow{\mathsf{merge}(r_1,r_2)} C[v_m \mapsto (\mathsf{merge}(s_T, s_1, s_2), E_1 \cup E_2); r_1 \mapsto v_m; P(v_m) \mapsto [v_1, v_2]]}$$

Figure 2: Raw operational semantics. Fork creates a child of an existing replica head and copies its snapshot; Query is read-only.

Here $\mathsf{GCA}_P(v_1, v_2) = v_T$ means that v_T is a common ancestor reached by every other common ancestor. It is unique if it exists. A history may instead have several incomparable maximal common ancestors; ordinary Merge is then unavailable, and Section 6 uses all of those maximal merge bases to construct a virtual base.

The initial configuration has one active replica, numbered 0, at version 0. Fork creates a fresh child of the source head, with the same state and event set and with the source version as its sole parent. Apply adds visibility edges from the issuer's entire current event set to the new event. Merge writes its result at r_1 and leaves r_2 unchanged. Thus fork preserves shared history, merge is not broadcast, and neither fork nor merge introduces an event or visibility edge. Version numbers also serve as topological ranks. The premises $v < v'$ in Fork and $v_1, v_2 < v_m$ in Merge preserve the condition that every parent has a smaller number than its child; they do not order application events.

3 Why the earlier induction fails

The published bottom-up induction [3] derived one replay order from every event in the configuration, then used that order to explain the smaller event set stored at an old version. Consider concurrent, noncommuting events e_1, e_2 and a later event e_3 with e_2 vis e_3. The global order sees e_3 and removes the edge $e_1 \to e_2$. An old version containing only $\{e_1, e_2\}$ then admits both replay orders even though the two folds produce different states.

Proposition 3.1 (global-order convergence is false)**.** *There is a two-operation add-wins signature and a legal configuration with a backward-closed event set E such that two enumerations of E both respect the configuration-global arbitration relation but fold to different states.* `convergence_over_backward_closed_subsets_false`

The checked countermodel is in `Metatheory/Join/Convergence_CounterModel.lean`. The defect is not an exotic property of that datatype: it is a mismatch between the events used to define the order and the events being replayed.

The repair defines the replay order relative to both the configuration C and the set E under consideration. We write ord for an independently selected full-event replay order. The submission manuscript reserves rc for its operation-level conflict relation and event lift; the event order here need not be that lift. Fix a proof-local event replay order $\mathsf{ord}(e_1, e_2) \in \{\mathsf{first}, \mathsf{second}, \mathsf{either}\}$. Write $e_1 \parallel_C e_2$ when neither event sees the other, $e_1 \rightleftarrows e_2$ when their updates commute, and $e_1 \ \mathsf{ord}\ e_2$ as an abbreviation for $\mathsf{ord}(e_1, e_2) = \mathsf{first}$. The set-relative replay relation is then

$$\begin{aligned} e_1 \ \mathsf{lo}_{C,E}\ e_2 \quad \Longleftrightarrow \quad & \big(e_1 \ \mathsf{vis}\ e_2 \wedge (e_1 \ \mathsf{ord}\ e_2 \vee e_2 \ \mathsf{ord}\ e_1)\big) \\ & \vee \big(e_1 \parallel_C e_2 \wedge e_1 \ \mathsf{ord}\ e_2 \wedge \neg\exists e_3 \in E.\ e_2 \ \mathsf{vis}\ e_3 \wedge (e_2 \ \mathsf{ord}\ e_3 \vee e_3 \ \mathsf{ord}\ e_2)\big). \end{aligned}$$

In the Lean sources, this relation is `loOn C E`. Only an absorber inside E may erase an edge needed to replay E. We reserve lo_C for the configuration-global relation `lo C`, whose absorber existential ranges over every event in C.

The set-relative order repairs the scope of arbitration, but the published bottom-up construction has a separate failure: state-correct witnesses for the two inputs need not expose an event that can be peeled through merge.

Example 3.2 (The defeater)**.** Let X be the element type and fix $x \in X$. In the published add-wins OR-set case study [3], a state is a finite set $S \subseteq \mathbb{T} \times X$. An event $(t, r, \mathsf{add}(x))$ inserts the uniquely timestamped record (t, x). An event $(t, r, \mathsf{rem}(x))$ removes every record (t', x) currently in the state, regardless of t'. A remove and an add of the same element therefore do not commute; when they are concurrent, the event replay order orders the remove first, giving add-wins behavior. The three-way merge is

$$\mathsf{merge}(l, a, b) = (l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l).$$

Let

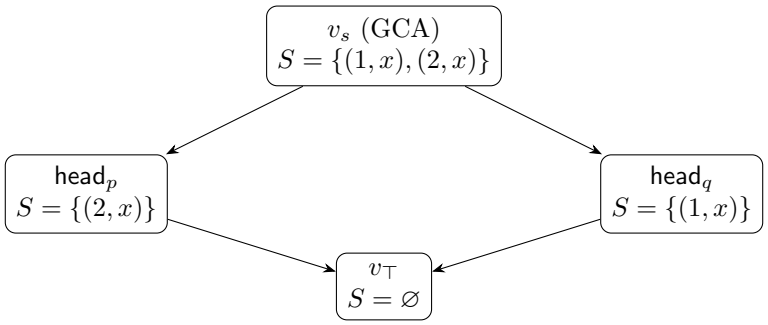
$$A_p = (1, p, \mathsf{add}(x)), \quad A_q = (2, q, \mathsf{add}(x)), \quad R_p = (3, p, \mathsf{rem}(x)), \quad R_q = (4, q, \mathsf{rem}(x)).$$

Replica q first forks from the initial version. Replicas p and q then issue A_p and A_q, respectively. A staging replica s forks from p's one-add version and merges q's one-add version, producing v_s with $E(v_s) = \{A_p, A_q\}$. Replica p issues R_p while its causal past is only $\{A_p\}$, then merges with v_s. Symmetrically, q issues R_q with causal past $\{A_q\}$, then merges with v_s. Figure 3 contracts the snapshot-preserving versions introduced by the two forks.

The final merge is GCA-legal: its GCA is v_s, and

$$E(v_s) = \{A_p, A_q\} = E(\mathsf{head}_p) \cap E(\mathsf{head}_q).$$

Writing $S(v)$ for the OR-set state at version v, the merge calculation is



Thus every state-correct replay of p's head places R_p before A_q and ends in A_q; symmetrically, every state-correct replay of q's head ends in A_p. A replay of the final union that produces the empty state must instead end in a remove. But applying $\mathsf{rem}(x)$ always produces a state containing no record for x, whereas both side heads contain one. Neither head is therefore in the image of $\mathsf{rem}(x)$, so the bottom-up rules cannot peel the required final event.

The merged version itself does have a valid witness: $[A_p, R_p, A_q, R_q]$ folds to $\varnothing$. Its restriction to q's events is $[A_p, A_q, R_q]$, which also folds to $\varnothing$, not to $S(\mathsf{head}_q) = \{(1, x)\}$. The merged witness therefore cannot be assembled from state-correct side witnesses, which is the step required by the published bottom-up induction.

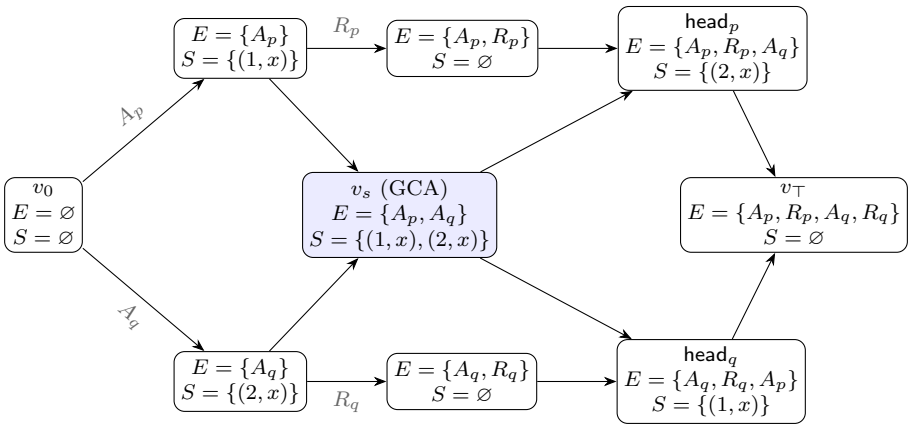


Figure 3: The OR-set defeater. Each shown version gives its event set E and concrete OR-set state S; snapshot-preserving fork versions are contracted. The shaded staging version v_s realizes the intersection of the two head event sets, so the final merge is GCA-legal. The removes R_p and R_q are buried inside their respective side witnesses.

Associativity, commutativity, and idempotence of merge do not repair the proof. A different checked countermodel satisfies the core update laws and a bounded join-semilattice merge but violates both the peel laws and the Join lemma. `coreVCs_lattice_insufficient`

These examples establish three boundaries: arbitration must be local to the set being linearized, a merged witness cannot in general be assembled by peeling state-correct side witnesses, and convergence needs a contextual bridge between merge and update history.

4 Internal replay adequacy

4.1 Canonical states

For a list $\pi = [e_1, \ldots, e_n]$, let $\mathsf{perm}(\pi, E)$ mean that π enumerates exactly E without duplicates. Let $\mathsf{respects}(\pi, R)$ mean that no later list element is required by R to precede an earlier element. Canonicality is then the following state predicate:

$$\mathsf{Canonical}(C, E, s) \iff \exists \pi.\ \mathsf{perm}(\pi, E) \wedge \mathsf{respects}(\pi, \mathsf{lo}_{C,E}) \wedge \mathsf{fold}(\sigma_0, \pi) = s. \quad \text{(Canonical)}$$

This is `IsCanonicalState`. The update-layer lemmas prove that any two such enumerations have the same fold under the supplied update conditions.

Definition 4.1 (canonical configuration)**.** $\mathsf{CanonicalConfig}(C)$ holds when every registered version contains a canonical state of its event set, visibility is transitive and irreflexive, and every stored event set is supported and causally closed. `CanonicalConfig`

The definition ranges over all versions, not only replica heads, because later merges may read an older version as their GCA.

Canonicality also yields a witness for the configuration-global replay relation lo_C, because $\mathsf{lo}_C|_{E\times E} \subseteq \mathsf{lo}_{C,E}$:

$$\begin{aligned}\mathsf{ReplayWitness}(C) \iff\ & \forall v, s, E.\ S(v) = (s, E) \Rightarrow \\ & \exists \pi.\ \mathsf{perm}(\pi, E) \wedge \mathsf{respects}(\pi, \mathsf{lo}_C) \wedge \mathsf{fold}(\sigma_0, \pi) = s.\end{aligned} \tag{Replay}$$

This per-version replay theorem is an internal adequacy result. It explains an implementation state as a fold of its operations, but it does not yet prove that the fold is legal for clients, agrees with an independent abstract state, or preserves queries. Section 5 states the client-facing theorem that adds those requirements.

4.2 The ternary Join obligation

The hypotheses of Join are easy to understate, so we name them explicitly. Let $\mathsf{WF}(C)$ mean that vis is transitive and irreflexive. For an event set E, define

$$\begin{aligned}\mathsf{Supported}_C(E) &\equiv \forall e \in E.\ e \in C.\mathsf{events}, \\ \mathsf{WeakClosed}_C(E) &\equiv \forall a, b.\ a\ \mathsf{vis}\ b \wedge \neg(a \rightleftarrows b) \wedge b \in E \Rightarrow a \in E.\end{aligned}$$

Weak closure retains visible predecessors only when their updates do not commute. Full causal closure drops the noncommutation premise.

Definition 4.2 (contextual ternary Join)**.** $\mathsf{JoinAt}(D, C)$ holds when, for all E_1, E_2, s_0, s_1, s_2,

$$\begin{aligned}&\mathsf{WF}(C),\quad \mathsf{Supported}_C(E_1),\quad \mathsf{Supported}_C(E_2),\quad \mathsf{WeakClosed}_C(E_1),\quad \mathsf{WeakClosed}_C(E_2), \\ &\mathsf{Canonical}(C, E_1 \cap E_2, s_0),\quad \mathsf{Canonical}(C, E_1, s_1),\quad \mathsf{Canonical}(C, E_2, s_2)\end{aligned}$$

imply

$$\mathsf{Canonical}\left(C, E_1 \cup E_2, \mathsf{merge}(s_0, s_1, s_2)\right). \tag{Join}$$

`JoinAt`

Figure 4 summarizes the proof after both canonicality and its merge-preservation obligation have been defined.

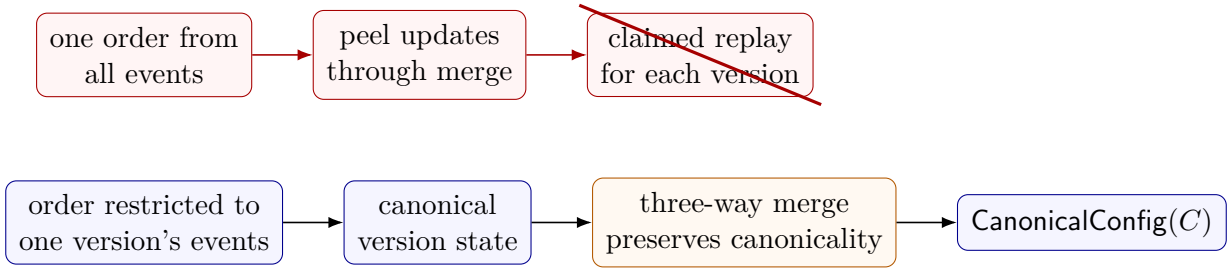


Figure 4: The refuted global-order argument and the repaired induction. The repaired path restricts ordering to one version's operations and asks each datatype to show that its three-way merge preserves the resulting state.

`Join` is $\forall C.\mathsf{JoinAt}(D, C)$. The restricted form $\mathsf{JoinOn}(D, G)$ is $\forall C.\, G(C) \Rightarrow \mathsf{JoinAt}(D, C)$. These definitions differ only in scope: `JoinAt` remains the single merge-preservation obligation. All-context Join supplies Join under every predicate, and $\mathsf{JoinOn}(D, \lambda_.\mathsf{True})$ is equivalent to $\mathsf{Join}(D)$. Section 5 explains how certified execution can establish the condition G.

Theorem 4.3 (ordinary replay adequacy)**.** *If a datatype satisfies* ***`Join`****, every configuration reachable from* ***`initConfig`*** *by* ***`Step`*** *has a valid replay witness at every registered version.* ***`replayWitness_of_join`***

The proof first establishes $\mathsf{CanonicalConfig}(C)$ and then obtains $\mathsf{ReplayWitness}(C)$ by projecting its canonical-state field and weakening $\mathsf{lo}_{C,E}$ to lo_C. The induction carries two invariants. `StoreInv` says that parents are allocated, event sets grow along parent edges, and every event has an allocated origin version that reaches every version containing it. The origin property proves the non-obvious half of $E_T = E_1 \cap E_2$: a shared event's origin reaches both branches, hence reaches their GCA. CanonicalConfig is the second invariant. Apply appends its fresh event to a canonical fold; merge invokes Join with the GCA equality; query and fork preserve both invariants.

The framework packages the eight-condition route as `CanonicalJoinLaws`. Its components are `JoinCoreLaws`, `FeasibleDeltaLaws`, and `CausalDeltaLaw`. Datatypes may construct this contract from stronger arbitrary-state equations or prove `Join` or `JoinOn` directly. A complete datatype certificate depends on the resulting merge property, not on one particular decomposition.

4.3 One reusable proof method for all-context Join

Join is the semantic convergence obligation. The framework offers a sufficient decomposition for delta-shaped ternary merges; an implementer may instead prove Join directly. We state the decomposition fully because its contextual premises are exactly what an arbitrary-state algebraic account loses.

Recall the proof-local event replay order ord fixed above. Write $e \# e'$ for distinct timestamps. For any relation R, write $R^+(a, b)$ when a nonempty finite R-path leads from a to b, and write

$$\mathsf{Acyclic}(R) \iff \forall a.\, \neg R^+(a, a).$$

The three `ReplayLaws` obligations are universally quantified over all states s, events, and finite event lists π:

$$\neg(e_1 \rightleftarrows e_2) \Rightarrow \big(\mathsf{ord}(e_1, e_2) = \mathsf{first} \vee \mathsf{ord}(e_2, e_1) = \mathsf{first}\big), \tag{U1}$$

$$\mathsf{Acyclic}\big(\lambda a\, b.\ \mathsf{ord}(a, b) = \mathsf{first}\big), \tag{U2}$$

$$\begin{aligned} e\#e' \wedge e\#e'' \wedge e'\#e'' \\ \wedge\, \mathsf{ord}(e, e') = \mathsf{first} \\ \wedge\, \big(\mathsf{ord}(e', e'') = \mathsf{first} \vee \mathsf{ord}(e'', e') = \mathsf{first}\big) \Rightarrow\ & \mathsf{do}(\mathsf{fold}(\mathsf{do}(\mathsf{do}(s, e'), e), \pi), e'') \\ = {}& \mathsf{do}(\mathsf{fold}(\mathsf{do}(\mathsf{do}(s, e), e'), \pi), e''). \end{aligned} \tag{U3}$$

(U1) requires coverage of concretely noncommuting pairs; ordered pairs may also commute. (U2) excludes every nonempty event-order cycle, including cycles on raw events with equal timestamps. It permits arbitrarily long chains. (U3) uses pairwise-distinct timestamps and an order-related successor, the same successor condition used by the absorber in $\mathsf{lo}_{C,E}$. These obligations belong to this reusable proof constructor, not to `MRDTSig` or the public client interface. `JoinCoreLaws` packages (U1)–(U3) and branch symmetry

$$\mathsf{merge}(l, a, b) = \mathsf{merge}(l, b, a). \tag{M1}$$

To state the remaining four obligations, define

$$\downarrow_C e = \{x \mid x = e \vee \big(\lambda a\, b.\ a \mathsf{\ vis\ } b \wedge \neg(a \rightleftarrows b)\big)^{+}(x, e)\}$$

The downset deliberately follows noncommuting visibility, rather than the event-order relation used by $\mathsf{lo}_{C,E}$. This is Lean's `visNC` and `downset`. Let $\mathsf{Adm}(C, E_1, E_2, e)$ abbreviate:

1. $\mathsf{WF}(C)$, support and weak closure of E_1, E_2;
2. $e \in E_1 \cup E_2$; and
3. union maximality, $\forall x \in E_1 \cup E_2.\ x \neq e \Rightarrow \neg(e\ \mathsf{lo}_{C,E_1 \cup E_2}\ x)$.

The feasible unit law is

$$\mathsf{Supported}_C(E) \wedge \mathsf{Canonical}(C, E, s) \Rightarrow \mathsf{merge}(\sigma_0, \sigma_0, s) = s. \tag{M2}$$

For the local redistribution law, assume $\mathsf{Adm}(C, E_1, E_2, e)$, $e \in E_1$, $e \notin E_2$, and

$$\begin{aligned} &\mathsf{Canonical}(C, E_1 \cap E_2, s_0), \quad \mathsf{Canonical}(C, \downarrow_C e \setminus \{e\}, B), \\ &\mathsf{Canonical}(C, E_1 \setminus \{e\}, t_1), \quad \mathsf{Canonical}(C, E_2, s_2). \end{aligned}$$

Then

$$\mathsf{merge}\big(s_0, \mathsf{merge}(B, t_1, \mathsf{do}(B, e)), s_2\big) = \mathsf{merge}\big(B, \mathsf{merge}(s_0, t_1, s_2), \mathsf{do}(B, e)\big). \tag{M3}$$

For shared redistribution, assume $\mathsf{Adm}(C, E_1, E_2, e)$, $e \in E_1 \cap E_2$, and canonical states

$$t_0 : (E_1 \cap E_2) \setminus \{e\}, \quad B : \downarrow_C e \setminus \{e\}, \quad t_1 : E_1 \setminus \{e\}, \quad t_2 : E_2 \setminus \{e\}.$$

The required equation is

$$\begin{aligned} &\mathsf{merge}\big(\mathsf{merge}(B, t_0, \mathsf{do}(B, e)), \mathsf{merge}(B, t_1, \mathsf{do}(B, e)), \mathsf{merge}(B, t_2, \mathsf{do}(B, e))\big) \\ &\quad = \mathsf{merge}\big(B, \mathsf{merge}(t_0, t_1, t_2), \mathsf{do}(B, e)\big). \end{aligned} \tag{M4}$$

Equations (M2)–(M4) form `FeasibleDeltaLaws`. "Feasible" is load-bearing: every state named above is a canonical state of the exact event set at that position in the proof.

Finally, `CausalDeltaLaw` considers any supported, weakly closed U and $\mathsf{lo}_{C,U}$-maximal $e \in U$. If

$$\mathsf{Canonical}(C, U \setminus \{e\}, A), \qquad \mathsf{Canonical}(C, \downarrow_C e \setminus \{e\}, B),$$

then

$$\mathsf{merge}(B, A, \mathsf{do}(B, e)) = \mathsf{do}(A, e). \tag{M5}$$

Theorem 4.4 (VC adequacy for replay)**.** *Obligations (U1)–(U3) and (M1)–(M5), equivalently* ***`CanonicalJoinLaws`****, imply* ***`Join`****, which in turn implies an internal replay witness for every version of every reachable raw configuration.* ***`CanonicalJoinLaws.join`****,* ***`replayWitness_of_join`***

Why these equations suffice. Choose a $\mathsf{lo}_{C,E_1 \cup E_2}$-maximal event. If it is local to one side, (M5) peels the inner causal delta, (M3) moves it through the outer merge, and the induction hypothesis joins the smaller event sets. If it is shared, (M5) peels it from the GCA and both branches, and (M4) moves the shared delta out once. (M2) closes the empty case; (M1) exchanges branches. The update laws justify swapping adjacent replay events and guarantee that the maximal-event decomposition is well founded. This is the checked proof structure behind `CanonicalJoinLaws.join`; the displayed obligations do not claim necessity.

5 Issuance, sequential meaning, and safety

RGA insertion illustrates why a plain signature still needs a client protocol. An insertion carries an anchor and a fresh identifier. Raw delivery can apply that record anywhere, but honest minting requires the issuer to have seen the anchor and to choose an identifier above its causal past. This is a historical fact about the minting state, not a shape predicate on an arbitrary later delivery state.

Definition 5.1 (issuance)**.** An `Issuance D` supplies the predicate

$$\mathsf{CanIssue} \quad : \mathsf{Op} \to \Sigma \to \mathsf{Prop}.$$

`IssuedStep` checks `CanIssue` only for an apply transition and checks it at the issuer's current materialized head. Non-apply transitions retain their raw semantics. Define the visible past of event e by $\mathsf{Past}_C(e) = \{x \in C.\mathsf{events} \mid x \ \mathsf{vis}\ e\}$. The persistent honesty assertion is

$$\mathsf{MintHonest}_I(C) \equiv \forall e \in C.\mathsf{events}.\ \exists \pi.\ \mathsf{perm}(\pi, \mathsf{Past}_C(e)) \wedge \mathsf{respects}(\pi, \mathsf{vis}) \wedge I.\mathsf{CanIssue}(e, \mathsf{fold}(\sigma_0, \pi)). \tag{Mint}$$

A final configuration does not store all former issuer heads. Consequently, `MintCertifiedReach` records that (Mint) held before and after every issued step rather than attempting to reconstruct a deleted origin state from the final heads. `MintCertifiedReachV` is the virtual-merge-base counterpart. Datatype proofs may derive persistent history properties from this witness.

$$\frac{C \xrightarrow{\mathsf{apply}(t,r,o)} C' \quad H(r) = v \quad S(v) = (s, E) \quad I.\mathsf{CanIssue}((t,r,o), s)}{C \underset{I}{\xRightarrow{\mathsf{apply}(t,r,o)}} C'} \textsc{Certified-Apply}$$

$$\frac{C \xrightarrow{\ell} C' \quad \ell \neq \mathsf{apply}(-,-,-)}{C \underset{I}{\xRightarrow{\ell}} C'} \textsc{Certified-Other}$$

Figure 5: Issuance-certified execution. Certification restricts minting at the issuer head; it does not alter raw delivery or merge semantics. A `MintCertifiedReach` trace additionally retains the historical mint witness needed after the issuer moves to a later head.

Definition 5.2 (sequential specification)**.** A `SequentialSpec D` supplies an abstract state, initial state, deterministic step, query function, and

$$\mathsf{Legal} : \mathsf{List}(\mathsf{Op}) \to \mathsf{Prop}.$$

The legality predicate sees only the abstract event history. It cannot inspect the implementation state. A total datatype sets it to true.

Definition 5.3 (the sole *ord* order)**.** A verified datatype supplies one `EventOrder`, called *ord*. For a version event set E, `loOn` retains a visibility edge exactly when *ord* classifies that pair as conflicting. It adds an edge between concurrent events when *ord* returns first-before-second, subject to the set-relative conflicting-absorber clause. Thus Either identifies a nonconflicting pair and contributes no edge; under the unconstrained all-Either policy, `loOn` is empty.

The sufficient algebraic replay route requires *ord* to be acyclic: no nonempty `Relation.TransGen` path returns to its starting event. Useful chains are allowed. Its coverage condition says that concretely noncommuting pairs must be ordered, not that ordered pairs

must fail to commute. Its conditional-swap law uses the event replay order *ord* absorber. These are sufficient proof obligations, not the meaning of *ord* or mandatory fields of the public certificate. All-commuting implementations support every acyclic *ord* under these laws.

LWW separates concrete and abstract commutation. Its concrete state stores the greatest timestamped write, and both update and merge compute a maximum, so its updates commute. Ordinary overwrite steps do not commute, and its sole public *ord* orders unequal writes by their timestamped keys. Sorting a witness by that key preserves the concrete fold, respects both visibility and the concurrent *ord* edges, and yields an ordinary overwrite-register execution with the stored value. Concretely, `replayOrder_order_acyclic` derives acyclicity from strict key increase, while `timestamp_chain` confirms that a two-edge chain is valid. The same timestamp policy is used by its concrete replay laws, Join theorem, and sequential certificate; there is no auxiliary empty policy for LWW.

Definition 5.4 (specification linearizability)**.** Given event replay order P, sequential specification *Spec*, and representation relation $\mathsf{Rel} \subseteq \Sigma \times \mathit{Spec}.\mathsf{State}$, $\mathsf{SpecLin}(D, P, \mathit{Spec}, \mathsf{Rel}, C)$ holds when every $S(v) = (s, E)$ admits a list π such that

$$\begin{aligned} &\mathsf{perm}(\pi, E), \qquad \mathsf{respects}(\pi, \mathsf{loOn}(P, C, E)), \\ &\mathit{Spec}.\mathsf{Legal}(\pi), \qquad \mathsf{Rel}(s, \mathit{Spec}.\mathsf{run}(\pi)), \\ &\forall q. \quad D.\mathsf{query}(s, q) = \mathit{Spec}.\mathsf{query}(\mathit{Spec}.\mathsf{run}(\pi), q). \end{aligned} \tag{Spec-Lin}$$

`IsSpecLinearizable`

The first line constrains the explanation's events and ordering. The second rules out a witness that the sequential ADT itself considers illegal. The third rules out a representation relation that relates states but fails to preserve observations. Internal replay linearizability establishes none of these extra clauses automatically.

Definition 5.5 (verified MRDT)**.** An implementer supplies:

- an issuance relation I;
- one event replay order P, exposed as *ord*;
- a convergence certificate proving internal replay linearizability for every I-certified virtual-merge-base execution;
- an independent sequential specification *Spec* and representation relation Rel; and
- one sequential-correctness certificate proving (Spec-Lin) from any ordinary or canonical-virtual certified execution and its replay theorem.

`VerifiedMRDT`

At this point the framework responsibilities can be summarized without anticipating undefined interfaces:

Layer	Framework supplies	Datatype supplies
Raw execution	version DAG, apply, merge, query	state, update, merge, query
Replay proof	canonical-state induction, virtual bases	merge-preservation proof
Client protocol	certified transition schema	issuance predicate
Client meaning	witness theorem schema	*ord* and sequential proofs
Commit GC	distributed protocol and refinement	—
State GC	trace-erasure theorem	physical collection protocol

Table 1: Division of responsibility after packaging a verified MRDT. Replay is an internal proof layer; client meaning comes from the independent sequential specification.

Definition 5.6 (production package)**.** A `PackagedMRDT` is a record (n, D, V) containing a display name n, a raw signature D, and a certificate V of type `VerifiedMRDT D`. The typed `Production.registry` contains these complete packages.

The public capstones `VerifiedMRDT.correct` and `VerifiedMRDT.correctV` prove (Spec-Lin) for ordinary and virtual-merge-base executions. A datatype may also supply a `SafetyCertificate` for a client invariant not implied by sequential correctness. The bounded counter, for example, proves that observable values remain non-negative by carrying a global history invariant.

6 Canonical virtual merge bases

Criss-cross histories can have several maximal common ancestors and no registered GCA. Choosing one maximal ancestor can lose events represented by another. The framework instead constructs a synthetic state by recursively folding the maximal-common-ancestor frontier.

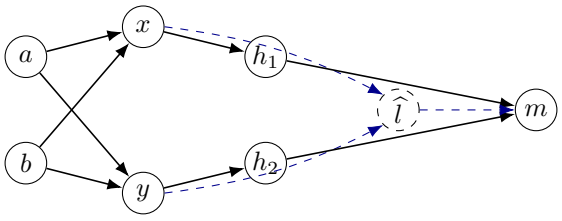


Figure 6: The registered DAG has two maximal common ancestors, x and y. The resolver constructs $S_{\widehat{l}} = \mathsf{foldMaximalBases}(\{x, y\})$ for event set $\mathcal{E}(x) \cup \mathcal{E}(y)$; only merge result m is allocated. Solid arrows are stored parent edges; blue dashed arrows describe the ghost computation.

`canonicalVirtualMergeBase` implements this resolver. The support decreases under the version order, which justifies the recursion. The key theorem `virtualMergeBaseState_canonical` proves that the synthetic state is canonical for the union of the maximal ancestors' event sets. The store theorem `maximalCommonAncestorsWithAny_events_cover` shows that this union covers every event shared by the two heads.

`StepV` conservatively embeds every ordinary `Step` and adds one virtual merge constructor. For $S(v_i) = (s_i, E_i)$ and resolver $\widehat{S}(C, v_1, v_2)$, that rule is

$$\frac{\begin{array}{c} H(r_i) = v_i \ (i = 1, 2) \quad S(v_i) = (s_i, E_i) \\ v_m \notin \mathsf{dom}(S) \quad v_1, v_2 < v_m \end{array}}{\begin{array}{l} C \xrightarrow{\mathsf{merge}(r_1, r_2)} C', \\ S'(v_m) = (\mathsf{merge}(\widehat{S}(C, v_1, v_2), s_1, s_2), E_1 \cup E_2), \\ H'(r_1) = v_m, \qquad P'(v_m) = [v_1, v_2] \end{array}} \textsc{Virtual-Merge}$$

It allocates only the result version. The maximal-common-ancestor frontier and its recursively merged state are ghost semantic values, not virtual commits that a runtime must store or later garbage-collect.

Theorem 6.1 (virtual-merge-base replay adequacy)**.** *The same* `Join` *used for ordinary execution gives every version reachable under* `StepV` *a valid internal replay witness when the framework uses the canonical resolver.* `replayWitnessV_of_join`

The proof first establishes canonicality of $\widehat{S}$ for the union of the maximal-common-ancestor event sets. It then applies the same Join premise used by ordinary merge. Therefore virtual merge bases introduce no new datatype merge law; they strengthen only the framework's construction of the base state.

7 Distributed commit-history garbage collection

Replicas fetch immutable commits asynchronously, so each replica must decide from its local holdings when history can be removed. The collector therefore runs as a local transition alongside ordinary fetch and head synchronization. Its specification baseline is the same asynchronous protocol with collection steps erased, allowing each collection step to refine a stutter while network steps retain their direct counterparts.

7.1 Minimal local state

For each replica, the logical collector stores

$$L = \{\mathsf{head} : \mathit{Version}, \quad \mathsf{commits} : \mathcal{P}(\mathit{Version})\}, \qquad \mathsf{head} \in \mathsf{commits}.$$

The parent relation, fixed roster, and immutable optional author of each version are protocol parameters. Apply-created versions may record their origin replica; roots and merge versions are unauthored. Authorship establishes progress evidence rather than authenticity against an adversary. An envelope carries a commit set. Fetch unions that set into the receiver's holdings, and head synchronization is a separate transition. `GC.Local`, `GC.Envelope`

A distributed world is a replica-indexed family of these local records:

$$W : \mathit{Replica} \to \mathit{Local}.$$

Frontier evidence is computed from these records and the fixed protocol parameters.

Evidence is derived rather than stored. Replica r has frontier evidence at L when L retains a commit authored by r that reaches $L.\mathsf{head}$. Formally,

$$\mathsf{evidence}(L, r) = \{v \in L.\mathsf{commits} \mid \mathsf{author}(v) = r \wedge v \preceq L.\mathsf{head}\}.$$

`GC.EvidenceComplete` requires such evidence for every other roster member. The negative control `GC.EvidenceSPOT.missing_author` confirms that an unauthored merge head cannot stand in for a missing author's evidence.

7.2 Collection certificate

A `GC.Certificate` supplies a keep set and a compressed reachability relation $\preceq_K$. It contains the completeness proof, retains the local head and one derived witness for every remote roster member, proves support in the local holding, and preserves exact reachability and GCA answers between retained versions. The canonical constructor `GC.Certificate.ofMaximalCommonAncestorClosed` takes K closed under maximal common ancestors and instantiates $\preceq_K$ with reachability in the original graph restricted to retained endpoints.

For local record L and keep set K, the certificate obligations are:

$$\begin{aligned}
&\mathsf{EvidenceComplete}(L), \qquad L.\mathsf{head} \in K, \qquad K \subseteq L.\mathsf{commits},\\
&\forall r \in \mathsf{roster}.\ r = \mathsf{self} \ \vee\ \exists v \in K.\ v \in \mathsf{evidence}(L, r),\\
&a, b \in K \Rightarrow \big(a \preceq_K b \iff a \preceq b\big),\\
&v_1, v_2, v_T \in K \Rightarrow \big(\mathsf{GCA}_K(v_1, v_2, v_T) \iff \mathsf{GCA}(v_1, v_2, v_T)\big).
\end{aligned}$$

The compressed graph connects two retained versions when the original graph reached between them. It need not retain intermediate paths or the old root. Figure 7 shows the distinction between retaining a path and retaining its reachability fact.

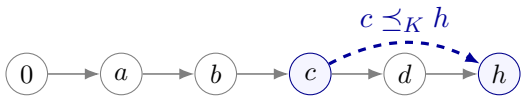


Figure 7: Compressed history retains endpoint answers, not the root or the intermediate path. Branching histories additionally retain the maximal common ancestors required by future GCA queries.

`GC.compressedReaches_iff` proves reachability equivalence, and `GC.root_absent_when_dropped` proves that a root outside the keep set is absent from the retained carrier. Keeping only the heads is insufficient: two retained heads may have a maximal common ancestor that collection would otherwise erase, changing the next merge base. Keeping the entire ancestor spine is sufficient but unnecessary; closure under maximal common ancestors is the smaller semantic condition.

7.3 Direct asynchronous refinement

The collecting protocol has fetch, head synchronization, and local GC. Its no-GC counterpart has the same network steps and no collection transition. The simulation relates one full and one compact local store by equal heads and commit-set inclusion:

$$\mathsf{StoreSim}(F, C) \ \equiv\ F.\mathsf{head} = C.\mathsf{head} \ \wedge\ C.\mathsf{commits} \subseteq F.\mathsf{commits} \ \wedge\ C.\mathsf{head} \in C.\mathsf{commits}.$$

$$\frac{M = \mathsf{commits}(W(s))}{W \longrightarrow W[d \mapsto \langle \mathsf{head}(W(d)), \mathsf{commits}(W(d)) \cup M\rangle]}\ \textsc{Fetch}$$

$$\frac{v \in \mathsf{commits}(W(r))}{W \longrightarrow W[r \mapsto \langle v, \mathsf{commits}(W(r))\rangle]}\ \textsc{Head-Sync}$$

$$\frac{\mathsf{Certificate}(W(r), K)}{W \longrightarrow W[r \mapsto \langle \mathsf{head}(W(r)), K\rangle]}\ \textsc{Collect}$$

Figure 8: Asynchronous commit-history GC. Fetch is the ordinary remote-fetch path and unions immutable commits. Head synchronization is explicit. Collect is local and requires complete frontier evidence plus reachability/GCA preservation for the keep set.

Let $\mathsf{NoGCStep}$ contain only Fetch and Head-Sync. The one-step simulation is

$$\mathsf{WorldSim}(F, C) \wedge C \longrightarrow_{\mathrm{GC}} C' \Longrightarrow \exists F'.\ F \longrightarrow_{\mathrm{NoGC}}^{0 \text{ or } 1} F' \wedge \mathsf{WorldSim}(F', C').$$

The collection case chooses $F' = F$; network cases take their matching no-GC step. Induction over the finite collecting trace gives the execution theorem.

Theorem 7.1 (distributed collection refines no-GC)**.** *Every finite collecting execution is matched by a finite no-GC execution. Fetch and head synchronization match their counterparts; collection stutters. The final worlds have equal replica heads, so head-state reads are unchanged.* *GC.execution_refines_noGC, GC.read_preserved*

This store theorem deliberately says only that an already-taken collecting trace has a no-GC match. To connect stores to the datatype semantics, define a proof-level runtime $R = \langle C, W \rangle$. Well-formedness says every locally held version is allocated in C, and every semantic head agrees with and is present in its local holding. A visible step additionally proves that its required versions are available: ordinary merge needs both heads and its GCA; a virtual merge needs the resolver's declared read set. Visible steps contain the actual `Step` or `StepV` derivation and install the new head in the local holding. Fetch and collection leave C unchanged.

Theorem 7.2 (runtime history-GC refinement)**.** *If* $R_0 \xRightarrow[\mathsf{runtime}]{\lambda_1 \ldots \lambda_n} R_n$*, then erasing silent fetch and collection labels yields*

$$R_0.C \xRightarrow[\mathsf{Step}]{\mathsf{filterMap}(\lambda_1 \ldots \lambda_n)} R_n.C.$$

The analogous result holds for `StepV` *and an explicit virtual-resolver read set.* *GC.runtime_refines_core, GC.runtime_refines_coreV*

The core configuration is ghost simulation state, not a second physical copy required of an implementation. The theorem also makes the collector's trust boundary concrete: malformed network data and Byzantine authorship are out of scope; commits and authorship are immutable protocol facts.

The refinement is a trace-safety result, not a theorem that collection preserves every transition that could have been taken later. A visible Fork requires the source head to be present in the source holding. Its store transition first receives the source's retained commits at the fresh destination and then installs the newly allocated child head. Fork therefore has the same shared-history shape after collection as it does in the raw semantics, while placing no special retention obligation on version 0.

8 Optional datatype-state garbage collection

Commit GC removes versions from a replica's history. It cannot decide whether an RGA tombstone, a Peritext mark boundary, or a TreeMove event can be removed from the state stored at a retained version. That decision is datatype specific.

Definition 8.1 (state-GC protocol)**.** A `StateGCProtocol D V` supplies

$$\begin{aligned} \mathsf{Physical} &: \mathsf{Type}, \\ \mathsf{semantic} &: \mathsf{Physical} \to \mathsf{Configuration}(D), \\ \mathsf{Valid} &: \mathsf{Physical} \to \mathsf{Prop}, \\ \mathsf{PhysicalStep} &: \mathsf{Physical} \to \mathsf{Option}(\mathsf{Label}) \to \mathsf{Physical} \to \mathsf{Prop}. \end{aligned}$$

It proves:

1. validity is invariant;

2. a silent physical transition leaves the semantic projection unchanged;
3. a visible transition refines a `StepV` transition.

Theorem 8.2 (state collection refines semantic execution)**.** *Erasing silent labels from any valid finite physical trace yields a finite* ***`StepV`*** *trace between the projected configurations.* ***`StateGCProtocol.refines`***

The combined physical state contains the datatype protocol state and the generic distributed commit stores. A combined transition is either a datatype physical step or a silent fetch/commit-GC step. Writing $P = \langle p, W \rangle$, the two cases are

$$\frac{p \xrightarrow{\lambda}_S p' \quad W \xrightarrow{\lambda}_{\mathsf{store}} W'}{\langle p, W \rangle \xrightarrow{\lambda} \langle p', W' \rangle}\ \textsc{Datatype} \qquad \frac{W \xrightarrow{\tau}_{\mathsf{fetch/GC}} W'}{\langle p, W \rangle \xrightarrow{\tau} \langle p, W' \rangle}\ \textsc{History}.$$

For $\lambda = \tau$, a datatype step leaves the commit store unchanged. For a visible label, its store evolution installs the version created by the semantic step. Both history transitions are silent at the datatype semantics.

Theorem 8.3 (the two collectors compose)**.** *For every valid finite combined trace, erasing state-GC, fetch, and commit-GC labels yields an execution of the uncollected virtual-merge-base semantics.* ***`GC.CombinedSteps.refinesV`***

The ordinary corollary `GC.CombinedSteps.refinesRaw` applies when every visible datatype transition proves a raw `Step`. Sided Peritext and TreeMove provide concrete state-GC protocols. Other datatypes may omit the interface entirely.

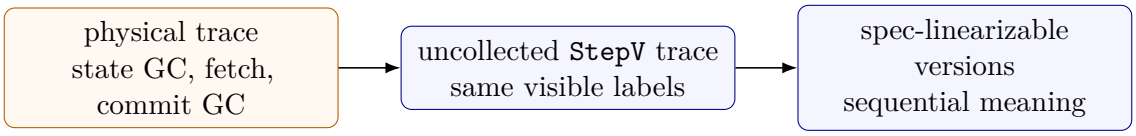


Figure 9: The two collectors remove different objects and compose by trace erasure. Commit GC removes versions; datatype GC removes metadata inside a retained state. The first arrow is `CombinedSteps.refinesV`; the second uses the `VerifiedMRDT` capstones.

9 Machine-checked instances

Table 2 lists the packages imported by `Metatheory/RefactorLedger.lean`. A checkmark means that the current Lean tree contains the named public package; it does not claim that the JavaScript runtime is extracted from that package.

The text instances begin with the replicated growable array (RGA), a sequence whose insertions name stable anchors and whose deletions retain identity-bearing tombstones [6]. EmbedRGA assigns immutable coordinates at creation; SidedEmbedRGA adds a left/right band used by its ordering policy. Peritext builds rich text from an underlying character sequence and formatting spans whose endpoints refer to stable character boundaries [7]. These packages make freshness and anchor liveness explicit in their issuance predicates. The observed-remove set has an ordinary finite-set specification: its *ord* orders a same-element remove before an add and leaves same-kind and cross-element operations as Either. The base tombstone RGA has an ordinary-list specification; it retains tombstones internally for concurrent integration but rejects minting after an observed anchor deletion. The bounded counter uses issuance to enforce available rights.

Datatype	Issuance	Convergence	Public spec	State GC
Bounded counter	✓	✓	✓	–
LWW register	✓	✓	✓	–
RGA	✓	✓	✓	–
EmbedRGA	✓	✓	✓	–
SidedEmbedRGA	✓	✓	✓	–
Observed-remove set	✓	✓	✓	–
Peritext/RichCore	✓	✓	✓	concrete protocol
TreeMove	✓	✓	✓	concrete protocol
AegisSheet	✓	✓	✓	certificate

Table 2: Current nontrivial production packages. Trivial-generation grow-only and counter products are also in the ledger. The AegisSheet sequential model is explained below.

The LWW package uses unconditional issuance. Its state is either unset or a timestamp–replica–value tuple; update and ternary merge take the tuple maximum. Its independent sequential specification is a total overwrite register over optional values. The checked witness construction sorts every version's writes by the tuple order, proves that this list respects loOn, and uses commutativity to preserve the stored maximum.

TreeMove represents a replicated tree whose nodes have stable identities and whose conflict resolution prevents concurrent moves from creating cycles; it follows the setting of highly available replicated-tree moves [8]. Its sequential state is the ordinary tree, not a copy of the replicated event set. AegisSheet models a collaborative spreadsheet with stable row, column, cell, and range identities and conflict policies for move, overwrite, range, and selective-undo operations [9]. Its sequential history is *causal-origin legal*: each operation must be valid in the causal context recorded when its issuer created it. Requiring validity after the entire preceding sequential prefix would reject two operations that were independently valid at concurrent origins.

The theorem `no_view_only_step` exhibits two reachable AegisSheet states with equal visible sheets whose responses to the same continuation differ. The independent sequential state therefore retains the causal identities needed to interpret later conflict handling, rather than reducing to visible cells alone.

External implementation correspondence. We validated the public AegisSheet Scala implementation against its published intent matrices and found four focused undo/range regressions at the audited revision. These are implementation-correspondence observations, not Lean counterexamples to the merge kernel. The reproducible audit and its revision boundary are in `docs/aegissheet-scala-audit.md`.

The separate negative ledger checks intentionally refuted or incomplete claims, including the MVR/single-register counterexample. MVR now has a public finite-live-set implementation and specification, with a value-set query. Writes discard their observed predecessors; ancestor-relative merge retains concurrent values without a grow-only write or overwrite log. Its combined theorem proves that precisely the causally maximal writes survive, and linear mint histories expose the last written value. The historical Queue control compares particular folds with unconditional pops, not every permitted linearization. Queue now has a public certificate for head-only FIFO replay with concurrent duplicate targets, timestamp-ordered surviving contents, and ordinary FIFO behavior on linear mint histories. FugueMax has a separate public package for its exact positional issuer and enriched implementation. Its abstract state is a plain list; immutable birth

metadata remains implementation-side. One certified-execution witness proves list correctness and maximal non-interleaving, including virtual merges, without a separate generation-model reachability premise. The release gate rejects `sorry` and `sorryAx`, requires every production entry to inhabit `PackagedMRDT`, and checks the distributed and composed GC capstones.

10 Related work

Conflict-free replicated data types obtain deterministic convergence without synchronous coordination by constraining state joins or delivered operations [5]. Replication-aware linearizability strengthens a bare convergence statement by asking for a sequential explanation of each replica state [4]. Our client theorem uses the same broad explanatory goal while making causality, the single *ord* policy, legality, abstract state, and observations explicit in the public datatype theorem.

MRDTs adapt replicated datatypes to branch-and-merge stores with ancestor-aware three-way merge [1]. Peepul verifies efficient MRDTs in F* using replication-aware simulation relations [2]. The bottom-up replay method subsequently proposed for automated MRDT verification [3] motivates our counterexample. Our correction is set-relative because a version may exclude events that exist elsewhere in the store, and it carries the invariant over every stored version because an old version may later serve as a merge base.

The instance proofs connect the generic result to established collaborative datatypes: RGA for ordered text [6], Peritext for rich-text formatting [7], highly available tree moves [8], and AegisSheet for spreadsheets [9]. The present artifact does not re-verify those papers wholesale. It verifies the signatures and sequential specifications in this repository and states any external scenario validation separately.

11 Reproduction and residual boundary

From the repository root, run the production proof and runtime gate with `./scripts/check-mrdt-refactor.sh`. Build both papers and the declaration ledger with `./scripts/check-working-papers.sh`.

The proved framework covers semantic MRDT executions, canonical virtual merge bases, distributed commit-history collection, optional state-GC protocols, and their composition. Same-replica crash recovery is not yet part of the verified or tested continuation protocol. Forking a fresh destination from an existing head is verified, but changing the fixed roster used by the collection protocol is not. The JavaScript implementation uses indexes and cached metadata for efficiency; the paper-facing logical GC state remains the minimal $\{\mathsf{head}, \mathsf{commits}\}$ record.

At the artifact boundary, Lean's `Configuration` stores the version store, replica heads, parent graph, and visibility relation used in Section 2. The functions `Configuration.headState` and `Configuration.headEvents` derive a replica's current observations from its head. Replay-only packages and alternative event-order hooks support migration and negative analysis in the artifact; production registry entries are the complete packages of Section 5.

12 Conclusion

Correctness for a branch-and-merge datatype cannot be proved by ordering the whole store and replaying arbitrary subsets of that order. The checked counterexample identifies the missing

locality, and the repaired induction orders exactly the operations of the version it explains. Three-way merge then has one precise task: preserve the resulting canonical state across the ancestor and two branches.

That replay theorem is deliberately not the final client claim. A verified package must also explain where operations could be created, which concurrent directions *ord* selects, which sequential histories are legal, how representation states correspond to abstract states, and why queries agree. The same separation clarifies reclamation: commit-history collection preserves the DAG answers required by asynchronous execution, while datatype-state collection preserves the semantic projection of retained versions. Their composition connects compact physical traces to the same client-facing theorem.

References

[1] Gowtham Kaki, Swarn Priya, K. C. Sivaramakrishnan, and Suresh Jagannathan. Mergeable replicated data types. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA):154:1–154:29, 2019. https://doi.org/10.1145/3360580.

[2] Vimala Soundarapandian, Adharsh Kamath, Kartik Nagar, and K. C. Sivaramakrishnan. Certified mergeable replicated data types. In *PLDI*, pages 332–347, 2022. https://doi.org/10.1145/3519939.3523735.

[3] Vimala Soundarapandian, Kartik Nagar, Aseem Rastogi, and K. C. Sivaramakrishnan. Automatically verifying replication-aware linearizability. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA1):871–897, 2025. https://doi.org/10.1145/3720452.

[4] Chao Wang, Constantin Enea, Suha Orhun Mutluergil, and Gustavo Petri. Replication-aware linearizability. In *PLDI*, pages 980–993, 2019. https://doi.org/10.1145/3314221.3314617.

[5] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *SSS*, pages 386–400, 2011. https://doi.org/10.1007/978-3-642-24550-3_29.

[6] Hyun-Gul Roh, Myeongjae Jeon, Jin-Soo Kim, and Joonwon Lee. Replicated abstract data types: Building blocks for collaborative applications. *Journal of Parallel and Distributed Computing*, 71(3):354–368, 2011. https://doi.org/10.1016/j.jpdc.2010.12.006.

[7] Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter van Hardenberg. Peritext: A CRDT for collaborative rich text editing. *Proceedings of the ACM on Human-Computer Interaction*, 6(CSCW2):531:1–531:36, 2022. https://doi.org/10.1145/3555644.

[8] Martin Kleppmann, Dominic P. Mulligan, Victor B. F. Gomes, and Alastair R. Beresford. A highly-available move operation for replicated trees. *IEEE Transactions on Parallel and Distributed Systems*, 33(7):1711–1724, 2022. https://doi.org/10.1109/TPDS.2021.3118603.

[9] Florian Pfeil, David Scandurra, and Julian Haas. AegisSheet: A compositional CRDT for collaborative spreadsheets. In *13th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*, pages 40–46, 2026.